

Hanoi University of Science and Technology

School of Information and Technology



CBUS: Modelling and Solution Approaches for the Capacitated Bus Routing Problem

Group Report

July 8, 2026

Abstract

This report addresses the Capacitated Pickup-and-Delivery Problem (CBUS), a complex routing challenge where a vehicle of capacity k must transport n passengers from pickup locations i to corresponding drop-off nodes $i + n$ starting from a central depot. To tackle this, we design and evaluate a comprehensive suite of optimization algorithms. For exact solutions, we formulate Constraint Programming (CP) and Integer Linear Programming (ILP) models, alongside a custom Branch-and-Bound search. To scale to larger instances, we implement heuristic and metaheuristic approaches, including greedy construction, local search, random walk, and Tabu Search. Through computational experiments, we analyze the practical trade-offs between exactness and scalability, highlighting which methods deliver the best balance of solution quality and execution time for varying problem sizes.

Contents

1	Introduction	4
1.1	Problem statement	4
1.2	Team work allocation	4
2	Mathematical Modelling	5
2.1	Sets, Parameters, and Notation	5
2.2	Standard Mathematical Model Formulation	5
2.3	Implementation Mapping Across Solution Families	7
2.4	Arc-Based Formulation (ILP)	7
3	Branch and Bound Optimization	9
3.1	State-Space Tree Exploration	9
3.2	Bounding Formulation and Critical Evaluation	9
3.3	Algorithmic Design	10
4	Constraint Programming (CP-SAT) Implementation	11
4.1	Modeling Paradigm and Variables	11
4.2	Constraint Channeling and Logical Propagation	11

4.3	Expected Performance Characteristics	12
4.4	Algorithmic Design	12
5	Integer Linear Programming Solution	14
5.1	Arc-Based Mathematical Formulation	14
5.2	Algorithmic Design and Model Generation	14
5.3	Alternative Design: Sequence-Based ILP Formulation	15
5.4	Structural Comparison: Arc-Based vs. Sequence-Based ILP	17
5.5	Expected Performance Characteristics	17
6	Heuristic and Metaheuristic Solutions	18
6.1	Methodological Motivation and Architecture	18
6.2	Constructive Phase: Greedy Nearest-Neighbor Heuristic	18
6.3	Exploitation Phase: Local Search via 2-Opt Vertex Swaps	19
6.3.1	O(1) Delta Cost Evaluation	20
6.4	Diversification Phase: Stochastic Destroy-and-Repair	21
6.5	Advanced Trajectory Optimization: Dynamic Tabu Search	22
6.5.1	Dynamic Tenure and Memory Management	22
6.5.2	Aspiration and Stagnation Diversification	22
7	Theoretical Complexity and Comparative Analysis	24
7.1	Performance Expectations across Dimensional Scales	25
8	Experimental Evaluation	26
8.1	Evaluation Methodology and Criteria	26
8.2	Phase 1: Local Benchmark Analysis ($N_i = 50$)	26
8.2.1	Exact Solvers: The Combinatorial Wall	27
8.2.2	Heuristic Trajectories: Speed vs. Quality	28

8.3	Phase 2: Large-Scale Operational Stress Test	28
8.3.1	Empirical Score Analysis	30
8.4	Conclusion	31
9	Final Conclusions	32
9.1	Report Synthesis	32
9.2	Key Academic and Engineering Takeaways	32
9.3	Practical Implications and Practical Realization	33
9.4	Future Research Horizons	33

1 Introduction

1.1 Problem statement

The CBUS problem asks for a shortest feasible route for one bus. The bus starts at the depot 0, visits every pickup and delivery node exactly once, and finally returns to 0. For each passenger $i \in \{1, \dots, n\}$, node i is the pickup location and node $i+n$ is the corresponding delivery location. The input also includes a distance matrix $c = (c_{ij})$, where c_{ij} is the travel cost from node i to node j .

A route is feasible only if it satisfies three requirements. First, every pickup and delivery point is visited exactly once. Second, passenger i must be picked up before being delivered, so node i must appear before node $i+n$. Third, at every point in the route, the bus load must be between 0 and capacity k .

The objective is to minimize the total travel distance:

$$\min c_{0,x_1} + \sum_{t=1}^{2n-1} c_{x_t,x_{t+1}} + c_{x_{2n},0}, \quad (1)$$

where $(x_1, x_2, \dots, x_{2n})$ is the ordered sequence of visited pickup and delivery nodes.

1.2 Team work allocation

The following table represent the contributions of each member to the project.

Member	Student ID	Main responsibility
Nguyen Hien Minh	202516670	Branch and bound solutions
Le Tri Cuong	202516646	Leader + OR-Tools SCIP/ILP solution + benchmark test + report + slide presentations
Nguyen Vu Hoang Anh	202516641	OR - Tools CP-SAT solution + Standard mathematical model
Dinh Khanh Nam	202516672	Greedy, Local search + Random walk, Tabu search solutions

2 Mathematical Modelling

Formulating a rigorous mathematical model constitutes a fundamental milestone in addressing any combinatorial optimization problem. The formulation presented below serves as the unified architectural foundation for three of our primary methodologies: it acts as the direct declarative model for Constraint Programming (CP-SAT), defines the implicit state-space boundaries for the Branch-and-Bound algorithm, and dictates the strict feasibility rules for our heuristic and metaheuristic trajectory searches. It is important to note that Integer Linear Programming (ILP), constrained by its strict requirement for algebraic linearization, necessitates a distinctly different structural formulation, which will be detailed in a subsequent section.

2.1 Sets, Parameters, and Notation

Let $\mathcal{V} = \{0, 1, \dots, 2n\}$ be the set of all vertices, where node 0 represents the central depot. We define $P = \{1, \dots, n\}$ as the set of pickup nodes and $D = \{n + 1, \dots, 2n\}$ as the set of delivery nodes, such that $\mathcal{S} = P \cup D$. Each passenger $i \in P$ must be picked up at node i and delivered to node $i + n$.

The bus has a maximum capacity of k passengers. Let c_{ij} denote the travel distance from node i to node j . The change in the vehicle's passenger load upon servicing node v is formalized as:

$$q_v = \begin{cases} 1, & \text{if } v \in P, \\ -1, & \text{if } v \in D, \\ 0, & \text{if } v = 0. \end{cases} \quad (2)$$

2.2 Standard Mathematical Model Formulation

The sequence-based formulation explicitly models the route as an ordered sequence of visited nodes. This perspective serves as the universal mathematical foundation for our Constraint Programming (CP), Branch-and-Bound, and Metaheuristic approaches, as they all construct or manipulate exact route orderings.

Decision Variables

- $s_t \in \mathcal{V}$: the specific node visited at step t , for $t = 0, \dots, 2n + 1$.
- $\pi_v \in \{1, \dots, 2n\}$: the sequence position (index) at which node v is visited, for $v \in \mathcal{S}$.
- $l_t \in \{0, \dots, k\}$: the load of the bus immediately after completing step t .

Constraints

1. Depot Initialization and Termination:

$$s_0 = 0, \quad s_{2n+1} = 0, \quad (3)$$

$$l_0 = 0, \quad l_{2n+1} = 0. \quad (4)$$

Justification: The bus must strictly originate from and return to the depot. The vehicle must depart empty and return empty, guaranteeing all onboarded passengers are eventually delivered.

2. Visit-Once and Mapping Requirements:

$$\{s_1, s_2, \dots, s_{2n}\} = \mathcal{S} \quad (5)$$

$$s_t = v \iff \pi_v = t \quad \forall v \in \mathcal{S}, \forall t \in \{1, \dots, 2n\} \quad (6)$$

Justification: Equation (3) ensures the intermediate sequence forms a perfect permutation of all service nodes, preventing duplicates or omissions. Equation (4) strictly couples the node-at-step variables (s_t) with the step-of-node variables (π_v), allowing bidirectional chronological reasoning.

3. Precedence Constraints:

$$\pi_i < \pi_{i+n} \quad \forall i \in P \quad (7)$$

Justification: A core CBUS requirement: a passenger's pickup location must chronologically precede their delivery location.

4. Capacity and Load Transitions:

$$l_t = l_{t-1} + q_{s_t} \quad \forall t \in \{1, \dots, 2n+1\} \quad (8)$$

$$0 \leq l_t \leq k \quad \forall t \in \{0, \dots, 2n+1\} \quad (9)$$

Justification: The dynamic load l_t is updated sequentially based on the nature of the visited node s_t . The domain restriction in (7) intrinsically prevents system overloading (exceeding k) and logical impossibilities (negative passengers).

Objective Function

The overarching goal is to minimize the total sequential travel distance:

$$\min \sum_{t=0}^{2n} c_{s_t, s_{t+1}} \quad (10)$$

2.3 Implementation Mapping Across Solution Families

While the formulation above is universally abstract, its practical implementation adapts to the underlying logic of each solver:

- **Constraint Programming (CP-SAT):** Variables s_t and π_v map to integer variables. The permutation constraint (3) is efficiently enforced via the `AllDifferent` global constraint. The objective distances $c_{s_t, s_{t+1}}$ are extracted utilizing `Element` constraints over a flattened cost matrix.
- **Branch and Bound:** The sequence s_t represents the recursive decision tree depth. Precedence (5) and capacity (7) are utilized as aggressive pruning criteria to eliminate invalid state-space branches.
- **Heuristic/Metaheuristic Methods:** The sequence s_t is maintained as an array data structure. Transition operations (e.g., 2-opt swaps, insertions) evaluate sequence validity strictly against equations (3), (5), and (7) via an evaluation function (`is_valid`) before acceptance.

2.4 Arc-Based Formulation (ILP)

In contrast to the sequential sequence-based model, Integer Linear Programming (ILP) requires an algebraic, arc-based formulation. It relies on binary decision variables to model network flow:

$$y_{ij} = \begin{cases} 1, & \text{if the bus travels directly from node } i \text{ to node } j, \\ 0, & \text{otherwise} \end{cases} \quad i, j \in \mathcal{V}, i \neq j. \quad (11)$$

The objective remains minimizing total distance, expressed linearly:

$$\min \sum_{i \in \mathcal{V}} \sum_{j \in \mathcal{V}, j \neq i} c_{ij} y_{ij} \quad (12)$$

Subject to routing degree constraints:

$$\sum_{j \in \mathcal{V}, j \neq i} y_{ij} = 1 \quad \forall i \in \mathcal{V} \quad (13)$$

$$\sum_{j \in \mathcal{V}, j \neq i} y_{ji} = 1 \quad \forall i \in \mathcal{V} \quad (14)$$

To ensure CBUS feasibility and eliminate disjoint subtours, continuous timing variables t_i

and load variables l_i are integrated using Big-M linearizations:

$$t_0 = 0, \quad l_0 = 0 \tag{15}$$

$$t_i < t_{i+n} \quad \forall i \in P \tag{16}$$

$$t_j \geq t_i + 1 - M_t(1 - y_{ij}) \quad \forall i \neq j, j \neq 0 \tag{17}$$

$$l_j \leq l_i + q_j + M_l(1 - y_{ij}) \quad \forall i \neq j, j \neq 0 \tag{18}$$

$$l_j \geq l_i + q_j - M_l(1 - y_{ij}) \quad \forall i \neq j, j \neq 0 \tag{19}$$

$$0 \leq l_i \leq k \quad \forall i \in \mathcal{V} \tag{20}$$

Justification: Equations (11) and (12) ensure continuous route flow (one predecessor, one successor). Equations (15) through (17) act as conditional logical gates; when arc (i, j) is active ($y_{ij} = 1$), the Big-M terms neutralize, forcing strict sequential updates for time and load. This formulation provides strict algebraic bounds but introduces significant computational overhead via $O(n^2)$ binary variables compared to the sequence-based approach.

3 Branch and Bound Optimization

3.1 State-Space Tree Exploration

The Branch-and-Bound (B&B) framework computes exact global solutions for the CBUS problem by recursively constructing the routing sequence. At each decision depth t (representing the chronological step in the tour), the algorithm systematically branches by selecting an unvisited candidate vertex $v \in \mathcal{S}$ to append to the partial trajectory.

To ensure semantic validity during the state-space tree traversal, the decision space is dynamically pruned via a localized feasibility check before deeper recursion is allowed. A candidate node v is declared valid at step t if and only if it satisfies the following multi-layered constraints simultaneously:

- **Vertex Uniqueness:** v has not been visited in any ancestral state of the current branch.
- **Precedence Consistency:** If $v \in D$ (a delivery destination), its corresponding pickup pair $v - n \in P$ must exist within the ancestral visited set.
- **Capacity Sufficiency:** If $v \in P$ (a pickup source), the current vehicle load prior to entry must satisfy $l_{t-1} < k$.

3.2 Bounding Formulation and Critical Evaluation

Let $c_{\min} = \min\{c_{ij} \mid c_{ij} > 0, i, j \in \mathcal{V}\}$ represent the global absolute minimum positive edge cost within the distance matrix. Suppose the recursive search has reached a depth t , establishing a partial route distance accumulated up to the current node. Since the vehicle must execute exactly $2n - t + 1$ subsequent edge transitions to service all remaining clients and return to the depot, we formulate a mathematically safe Lower Bound (LB) as:

$$LB = \sum_{m=0}^{t-1} c_{s_m, s_{m+1}} + (2n - t + 1)c_{\min} \quad (21)$$

Let z_{best}^* define the objective value of the current incumbent (the best feasible global solution discovered so far). If the condition $LB \geq z_{\text{best}}^*$ evaluates to true, the sub-tree rooted at the current node is immediately pruned.

Critical Analysis on Bound Tightness: While this bounding function is mathematically safe (guaranteeing that the global optimum will never be inadvertently discarded), it exhibits an inherent structural weakness. Because $c_{\min}$ is a static global scalar, it fails to capture the localized geometric distribution of the remaining unvisited vertices. In dense distance matrices or high-dimensional instances where $n \geq 15$, this bound becomes overly optimistic (loose), causing the algorithm to experience a severe combinatorial explosion as the pruning

frequency drops drastically. Consequently, while useful as an exact baseline verification tool, its scalability is naturally bottlenecked compared to metaheuristic strategies.

3.3 Algorithmic Design

The systematic implementation of the state-space exploration, bound evaluation, and back-tracking routine is formally detailed in Algorithm 1.

Algorithm 1: Branch-and-Bound Exact Sequence Optimizer for CBUS

Input: Passenger count n , vehicle capacity k , distance matrix c_{ij}
Output: Optimal route sequence s^* , minimum exact distance z_{best}^*

```

1  $c_{\min} \leftarrow \min\{c_{ij} \mid c_{ij} > 0\};$ 
2  $s_0 \leftarrow 0, z_{best}^* \leftarrow +\infty, l \leftarrow 0, z_{curr} \leftarrow 0;$ 
3  $\mathcal{V} \leftarrow \emptyset$  ; // Initialize the set of visited service nodes
4 function ExploreSequence( $t$ ):
5   foreach  $v \in \mathcal{S}$  do
6     if  $v \notin \mathcal{V}$  and IsFeasible( $v, l, \mathcal{V}$ ) then
7        $s_t \leftarrow v;$ 
8        $\mathcal{V} \leftarrow \mathcal{V} \cup \{v\};$ 
9        $z_{curr} \leftarrow z_{curr} + c_{s_{t-1}, v};$ 
10       $l \leftarrow l + q_v;$ 
11       $LB \leftarrow z_{curr} + (2n - t + 1)c_{\min};$ 
12      if  $LB < z_{best}^*$  then
13        if  $t = 2n$  then
14           $z_{total} \leftarrow z_{curr} + c_{s_{2n}, 0};$ 
15          if  $z_{total} < z_{best}^*$  then
16             $z_{best}^* \leftarrow z_{total};$ 
17             $s^* \leftarrow \text{Copy}(s);$ 
18          else
19            ExploreSequence( $t + 1$ );
20       $l \leftarrow l - q_v;$ 
21       $z_{curr} \leftarrow z_{curr} - c_{s_{t-1}, v};$ 
22       $\mathcal{V} \leftarrow \mathcal{V} \setminus \{v\};$ 
23 ExploreSequence(1);
24 return  $s^*, z_{best}^*;$ 

```

4 Constraint Programming (CP-SAT) Implementation

4.1 Modeling Paradigm and Variables

The Constraint Programming (CP) approach translates the CBUS routing challenge into a pure discrete sequence assignment problem. Unlike traditional mathematical programming which relies heavily on numerical relaxations, the CP-SAT solver operates natively on bounded integer domains and logical propagation.

To bridge the conceptual formulation with the solver’s architecture, we declare four explicit sets of decision variables:

- $point_t \in \{0, \dots, 2n\}$: Indicates the specific node visited at chronological step t , defined over $t \in \{0, \dots, 2n + 1\}$.
- $index_v \in \{1, \dots, 2n\}$: Serves as the position tracker, detailing the exact step at which service node v is visited.
- $customer_t \in \{0, \dots, k\}$: Represents the active passenger load within the vehicle immediately following step t .
- $step_dist_t \in \mathbb{Z}_{\geq 0}$: Stores the computed travel distance between the node at step t and step $t + 1$.

4.2 Constraint Channeling and Logical Propagation

The strength of the CP model lies in establishing explicit domain relationships between variables to accelerate search space pruning.

1. Depot and Uniqueness Rules: The route strictly originates and terminates at the depot ($point_0 = point_{2n+1} = 0$), maintaining zero initial and final passenger loads. To enforce the strict visit-once policy, we utilize the global logical constraint:

$$\text{AllDifferent}(point_1, \dots, point_{2n}) \quad (22)$$

which guarantees the sequence forms a complete permutation of all service requests.

2. Inverse Mapping (Bijection) Implementation: A critical mechanism in our implementation is establishing a strong bidirectional link between the step-centric view ($point_t$) and the node-centric view ($index_v$). As outlined in the core algorithm, this is achieved via auxiliary Boolean indicators ($b_{t,v}$):

$$b_{t,v} \iff (index_v = t) \wedge (point_t = v) \quad \forall t, v \in \{1, \dots, 2n\} \quad (23)$$

This channeling allows the solver to implicitly satisfy the pickup-before-delivery precedence rule ($index_i < index_{i+n}$) and propagates bound reductions simultaneously across both arrays.

3. Load Evolution via Implications: To track bus capacity, we inject logical implications using reified Booleans (`pickup_t` and `delivery_t`). The load transition follows a state-machine logic:

$$(1 \leq point_t \leq n) \implies customer_t = customer_{t-1} + 1 \quad (24)$$

$$(n < point_t \leq 2n) \implies customer_t = customer_{t-1} - 1 \quad (25)$$

Because the domain of $customer_t$ is strictly bounded within $[0, k]$, any variable assignment that causes overload or negative loads is instantly invalidated by the solver.

4. Matrix Linearization (Flattening): The objective function requires evaluating the distance cost $\sum step_dist_t$. Since CP-SAT natively evaluates 1D element arrays, we flatten the 2D distance matrix D into a 1D vector. The exact travel distance at step t is extracted via an `Element` constraint utilizing the interpolated index:

$$matrix_index_t = point_t \times (2n + 1) + point_{t+1} \quad (26)$$

$$step_dist_t = Flatten(D)[matrix_index_t] \quad (27)$$

4.3 Expected Performance Characteristics

The CP-SAT model provides a highly expressive and robust framework for the CBUS problem. Due to its advanced logical propagation and constraint channeling, it is exceptionally proficient at locating high-quality feasible solutions rapidly, even if full optimality cannot be explicitly proven. It functions excellently as an exact baseline for small to moderate dimensions. However, as n scales linearly, the necessity to generate $O(n^2)$ Boolean indicators for the inverse mapping and flattening operations induces significant memory and initialization overhead. Therefore, for extreme large-scale benchmarks, heuristic and metaheuristic pipelines are anticipated to exhibit superior time-to-solution ratios.

4.4 Algorithmic Design

The procedural generation of the CP-SAT formulation is formalized in Algorithm 2. Unlike constructive heuristics or branch-and-bound, this procedure does not solve the problem directly; rather, it systematically declares the decision space, injects the logical constraints, and delegates the highly-optimized combinatorial search to the underlying CP engine.

Algorithm 2: CP-SAT Declarative Model Generation for CBUS

Input: Passenger count n , vehicle capacity k , distance matrix D

Output: Optimal or feasible routing sequence $point^*$, objective cost z^*

```
1  $\mathcal{M} \leftarrow \text{InitializeEmptyModel}();$ 
   // 1. Variable Declarations
2 foreach  $t \in \{0, \dots, 2n + 1\}$  do
3    $\mathcal{M}.\text{AddIntVar}(point_t \in \{0, \dots, 2n\});$ 
4    $\mathcal{M}.\text{AddIntVar}(customer_t \in \{0, \dots, k\});$ 
5 foreach  $v \in \{1, \dots, 2n\}$  do
6    $\mathcal{M}.\text{AddIntVar}(index_v \in \{1, \dots, 2n\});$ 

   // 2. Boundary and Uniqueness Constraints
7  $\mathcal{M}.\text{Add}(point_0 = 0 \wedge point_{2n+1} = 0);$ 
8  $\mathcal{M}.\text{Add}(customer_0 = 0 \wedge customer_{2n+1} = 0);$ 
9  $\mathcal{M}.\text{AddAllDifferent}(point_1, \dots, point_{2n});$ 

   // 3. Channeling and Precedence Rules
10 foreach  $v \in \{1, \dots, 2n\}$  do
11   foreach  $t \in \{1, \dots, 2n\}$  do
12      $b_{t,v} \leftarrow \mathcal{M}.\text{AddBoolVar}();$ 
13      $\mathcal{M}.\text{AddImplication}(b_{t,v} \iff (index_v = t \wedge point_t = v));$ 

14 foreach  $i \in \{1, \dots, n\}$  do
15    $\mathcal{M}.\text{Add}(index_i < index_{i+n});$ 

   // 4. Capacity State Machine
16 foreach  $t \in \{1, \dots, 2n\}$  do
17    $\mathcal{M}.\text{AddImplication}(point_t \leq n \implies customer_t = customer_{t-1} + 1);$ 
18    $\mathcal{M}.\text{AddImplication}(point_t > n \implies customer_t = customer_{t-1} - 1);$ 

   // 5. Objective Function (Matrix Flattening)
19  $D_{\text{flat}} \leftarrow \text{FlattenMatrix}(D);$ 
20 foreach  $t \in \{0, \dots, 2n\}$  do
21    $m\_idx_t \leftarrow \mathcal{M}.\text{AddIntVar}(point_t \times (2n + 1) + point_{t+1});$ 
22    $\mathcal{M}.\text{AddElement}(m\_idx_t, D_{\text{flat}}, step\_dist_t);$ 
23  $\mathcal{M}.\text{Minimize}(\sum_{t=0}^{2n} step\_dist_t);$ 

   // 6. Execution and Extraction
24  $status, point^*, z^* \leftarrow \text{CPSolver.Solve}(\mathcal{M});$ 
25 return  $point^*, z^*;$ 
```

5 Integer Linear Programming Solution

5.1 Arc-Based Mathematical Formulation

The Integer Linear Programming (ILP) solution models the CBUS problem as an exact arc-selection challenge on a directed complete graph. We introduce a binary decision variable y_{ij} that evaluates to 1 if the route explicitly utilizes the directed arc from node i to node j , and 0 otherwise.

To eliminate invalid disconnected subtours and enforce the strict pickup-before-delivery precedence, continuous order variables t_i are utilized. Concurrently, continuous load variables l_i enforce the physical bus capacity strictly along the selected arcs. The objective is to minimize the total routing cost:

$$\min \sum_{i \in \mathcal{V}} \sum_{j \neq i} c_{ij} y_{ij} \quad (28)$$

The primary advantage of this classical vehicle-routing formulation is its mathematical clarity: every selected arc contributes its cost directly to the linear objective function without the need for complex variable transformations.

5.2 Algorithmic Design and Model Generation

The procedural generation of the exact arc-based ILP formulation is detailed in Algorithm 3. This setup leverages the SCIP backend to systematically construct the binary decision space, enforce the topological flow conservation rules (degree constraints), and append the Big- M linearizations for sequence and capacity tracking.

Algorithm 3: Arc-Based ILP/SCIP Model Generation for CBUS

Input: Passenger count n , vehicle capacity k , distance matrix c_{ij}
Output: Optimal route matrix y_{ij}^* , exact objective cost z^*

```
1  $\mathcal{M} \leftarrow \text{InitializeMILPSolver}(\text{Backend} = \text{'SCIP'});$   
   // 1. Variable Declarations  
2 foreach  $i, j \in \mathcal{V}$  where  $i \neq j$  do  
3    $y_{ij} \leftarrow \mathcal{M}.\text{AddBoolVar}();$   
4 foreach  $i \in \mathcal{V}$  do  
5    $t_i \leftarrow \mathcal{M}.\text{AddIntVar}(0, 2n + 1);$   
6    $l_i \leftarrow \mathcal{M}.\text{AddIntVar}(0, k);$   
  
   // 2. Flow Conservation (Degree Constraints)  
7 foreach  $i \in \mathcal{V}$  do  
8    $\mathcal{M}.\text{Add}\left(\sum_{j \neq i} y_{ij} = 1\right)$   $\triangleright$  Exactly one outgoing arc;  
9    $\mathcal{M}.\text{Add}\left(\sum_{j \neq i} y_{ji} = 1\right)$   $\triangleright$  Exactly one incoming arc;  
  
   // 3. Precedence and Initialization Rules  
10  $\mathcal{M}.\text{Add}(t_0 = 0 \wedge l_0 = 0);$   
11 foreach  $i \in P$  do  
12    $\mathcal{M}.\text{Add}(t_i < t_{i+n});$   
  
   // 4. Big-M Subtour Elimination and Capacity Tracking  
13  $M_t \leftarrow 2n + 3;$   $M_l \leftarrow k + 2;$   
14 foreach arc  $(i, j)$  with  $i \neq j$  and  $j \neq 0$  do  
15    $\mathcal{M}.\text{Add}\left(t_j \geq t_i + 1 - M_t(1 - y_{ij})\right);$   
16    $\mathcal{M}.\text{Add}\left(l_j - l_i - q_j \leq M_l(1 - y_{ij})\right);$   
17    $\mathcal{M}.\text{Add}\left(l_j - l_i - q_j \geq -M_l(1 - y_{ij})\right);$   
  
   // 5. Objective Function and Execution  
18  $\mathcal{M}.\text{Minimize}\left(\sum_{i \in \mathcal{V}} \sum_{j \neq i} c_{ij} y_{ij}\right);$   
19  $\text{status}, y_{ij}^*, z^* \leftarrow \mathcal{M}.\text{Solve}();$   
20 if status is OPTIMAL or FEASIBLE then  
21   return  $y_{ij}^*, z^*;$ 
```

5.3 Alternative Design: Sequence-Based ILP Formulation

For the purpose of rigorous structural evaluation, we present a secondary ILP formulation modeled identically to the step-by-step logic of Constraint Programming. Instead of selecting arcs, this model assigns nodes to chronological sequence steps via a binary assignment matrix $W_{s,v} \in \{0, 1\}$.

Algorithm 4: Sequence-Based ILP Model Generation (For Comparative Analysis)

Input: Passenger count n , vehicle capacity k , distance matrix c_{ij}
Output: Assignment matrix $W_{s,v}^*$, objective cost z^*

- 1 $\mathcal{M} \leftarrow \text{InitializeMILPSolver}(\text{Backend} = \text{'SCIP'});$
- // 1. Variable Declarations
- 2 **foreach** $s \in \{0, \dots, 2n+1\}$ and $v \in \mathcal{V}$ **do**
- 3 $W_{s,v} \leftarrow \mathcal{M}.\text{AddBoolVar}()$ ▷ 1 if step s visits node v ;
- 4 **foreach** $v \in \mathcal{V}$ **do**
- 5 $Pos_v \leftarrow \mathcal{M}.\text{AddIntVar}(0, 2n+1);$
- 6 **foreach** $s \in \{0, \dots, 2n+1\}$ **do**
- 7 $L_s \leftarrow \mathcal{M}.\text{AddIntVar}(0, k);$
- 8 $D_s \leftarrow \mathcal{M}.\text{AddNumVar}(0, \infty)$ ▷ Distance incurred at step s ;
- // 2. Boundary and Assignment Constraints
- 9 $\mathcal{M}.\text{Add}(W_{0,0} = 1 \wedge W_{2n+1,0} = 1 \wedge L_0 = 0);$
- 10 **foreach** $s \in \{1, \dots, 2n\}$ **do**
- 11 $\mathcal{M}.\text{Add}\left(\sum_{v \neq 0} W_{s,v} = 1\right)$ ▷ One node per intermediate step;
- 12 **foreach** $v \in \{1, \dots, 2n\}$ **do**
- 13 $\mathcal{M}.\text{Add}\left(\sum_{s=1}^{2n} W_{s,v} = 1\right)$ ▷ Each node visited exactly once;
- // 3. Position Mapping and Precedence
- 14 **foreach** $v \in \{1, \dots, 2n\}$ **do**
- 15 $\mathcal{M}.\text{Add}\left(Pos_v = \sum_{s=1}^{2n} s \cdot W_{s,v}\right);$
- 16 **foreach** $i \in P$ **do**
- 17 $\mathcal{M}.\text{Add}(Pos_i \leq Pos_{i+n});$
- // 4. Capacity and Tripartite Big-M Distance Linearization
- 18 $M \leftarrow 2 \cdot \max(c_{ij});$
- 19 **foreach** $s \in \{1, \dots, 2n+1\}$ **do**
- 20 $\mathcal{M}.\text{Add}\left(L_s = L_{s-1} + \sum_{v \in \mathcal{V}} q_v \cdot W_{s,v}\right);$
- 21 **foreach** $s \in \{0, \dots, 2n\}$ and $u, v \in \mathcal{V}$ where $u \neq v$ **do**
- 22 $\mathcal{M}.\text{Add}\left(D_s \geq c_{uv} - M \cdot (2 - W_{s,u} - W_{s+1,v})\right);$
- // 5. Objective Optimization
- 23 $\mathcal{M}.\text{Minimize}\left(\sum_{s=0}^{2n} D_s\right);$
- 24 $\text{status}, W_{s,v}^*, z^* \leftarrow \mathcal{M}.\text{Solve}();$

5.4 Structural Comparison: Arc-Based vs. Sequence-Based ILP

A critical modeling decision when developing integer programs for routing is choosing the fundamental decision variable architecture. The two paradigms formulated above demonstrate the dramatic impact of model structure on solver tractability.

The Fundamental Flaw of Sequence-Based ILP

While conceptually intuitive, the sequence-based formulation introduces a fatal structural flaw when calculating the objective function. To link the sequence variables to the distance matrix dynamically, we are forced to formulate an enormous cubic $O(n^3)$ block of tripartite Big- M constraints:

$$D_s \geq c_{uv} - M(2 - W_{s,u} - W_{s+1,v}) \quad \forall s, u, v \quad (29)$$

Critical Analysis: This formulation is a combinatorial nightmare. The continuous LP relaxation of these triple-variable Big- M constraints is inherently extremely weak. Because the bounds drop to zero during relaxation, the solver fails to extract a meaningful lower bound at the root node, forcing it to branch heavily and blindly. Consequently, the sequence-based ILP experiences severe stalling and computational timeouts even on trivial problem sizes ($n \leq 10$).

The Superiority of Arc-Based ILP

Conversely, the standard arc-based formulation utilizes binary edge variables y_{ij} , which directly explicitly maps to the cost matrix in the objective function without any conditional logic. The Big- M technique is strictly isolated to $O(n^2)$ constraints for subtour elimination and capacity tracking. The direct structural coupling of y_{ij} with c_{ij} produces a vastly tighter polyhedral envelope. It aligns fundamentally better with the Simplex and Branch-and-Cut algorithms powering modern MILP solvers, cementing its role as the definitive ILP architecture.

5.5 Expected Performance Characteristics

The Arc-Based ILP provides optimal, mathematically provable solutions for small-scale cases and serves as a highly reliable benchmark generator for moderate instances. Compared to CP-SAT, the arc variables natively support exact combinatorial optimization without channeling overhead. However, due to the inherent $O(n^2)$ spatial complexity of binary variables and the unavoidable presence of Big- M relaxations, memory allocation and branching times eventually become the bottleneck. For ultra-large-scale real-world scenarios, ILP transitions from an operational solver into an exact evaluation baseline.

6 Heuristic and Metaheuristic Solutions

6.1 Methodological Motivation and Architecture

Exact mathematical programming models (ILP and CP) guarantee absolute optimality but are fundamentally bounded by exponential computational complexities (NP -hard). For large-scale operational benchmarks, discovering a high-quality feasible solution within limited computational budgets is paramount. To solve this, we design a multi-stage metaheuristic framework.

Our architecture initiates with a fast constructive baseline, applies intensive trajectory-based local exploration using an optimized neighborhood evaluator, and culminates in a robust Tabu Search metaheuristic equipped with dynamic memory structures and stagnation-escape recovery mechanisms.

6.2 Constructive Phase: Greedy Nearest-Neighbor Heuristic

The optimization pipeline begins with a greedy constructive heuristic to establish an initial feasible upper bound. Starting from the central depot ($current_node = 0$), the algorithm sequentially builds the route vector by inspecting all unvisited candidates and selecting the vertex $v \in \mathcal{S}$ that minimizes the immediate edge cost $c_{current_node,v}$, provided that v satisfies the physical capacity and chronological precedence boundaries.

The programmatic procedure for this construction is detailed in Algorithm 5.

Algorithm 5: Greedy Nearest-Neighbor Construction Heuristic

Input: Passenger count n , vehicle capacity k , distance matrix c_{ij}

Output: A complete mathematically feasible route vector $\mathcal{R}$

```
1  $\mathcal{R} \leftarrow []$ ;
2  $visited \leftarrow [False] \times (2n + 1)$ ;
3  $load \leftarrow 0$ ,  $current\_node \leftarrow 0$ ;
4 for  $step \leftarrow 1$  to  $2n$  do
5    $best\_next \leftarrow -1$ ,  $min\_distance \leftarrow +\infty$ ;
6   foreach  $v \in \{1, \dots, 2n\}$  do
7     if not  $visited[v]$  then
8       // Check physical capacity and precedence constraints
9       if  $v \leq n$  and  $load \geq k$  then
10        continue;
11       if  $v > n$  and not  $visited[v - n]$  then
12        continue;
13       if  $c_{current\_node, v} < min\_distance$  then
14          $min\_distance \leftarrow c_{current\_node, v}$ ;
15          $best\_next \leftarrow v$ ;
16    $\mathcal{R}.Append(best\_next)$ ;
17    $visited[best\_next] \leftarrow True$ ;
18    $current\_node \leftarrow best\_next$ ;
19   // Dynamically update current vehicle load
20   if  $best\_next \leq n$  then
21      $load \leftarrow load + 1$ ;
22   else
23      $load \leftarrow load - 1$ ;
24 return  $\mathcal{R}$ ;
```

Complexity and Limitation Analysis: Since the algorithm evaluates all remaining unvisited nodes ($O(n)$) across $2n$ structural steps, its aggregate time complexity is strictly bounded by $O(n^2)$. The primary advantage of this heuristic is its execution speed. However, its core limitation is myopic (short-sighted) decision-making: a locally minimal edge choice early in the sequence often forces the vehicle into catastrophic edge costs during the final collection steps.

6.3 Exploitation Phase: Local Search via 2-Opt Vertex Swaps

To eliminate the localized inefficiencies inherited from the greedy seed, we implement an iterative improvement framework utilizing a *First-Improvement Local Search* over a 2-opt vertex exchange neighborhood.

6.3.1 $O(1)$ Delta Cost Evaluation

A major architectural feature of our implementation is the mitigation of the $O(n)$ evaluation bottleneck. Instead of naively recomputing the total route cost from scratch whenever a swap between positions i and j is proposed, the algorithm computes the exact cost variation (Δ) in constant $O(1)$ time by isolating the affected topological edges. The structural logic for this computation is formalized in Algorithm 6.

Algorithm 6: Constant-Time Incremental Cost Variation Evaluator

Input: Route vector $\mathcal{R}$, swap indices i and j (where $i < j$), distance matrix c_{ij}

Output: Exact cost delta Δ

```

1  $pre\_i \leftarrow \mathcal{R}[i - 1]$  if  $i > 0$  else 0;
2  $next\_i \leftarrow \mathcal{R}[i + 1]$  if  $i < |\mathcal{R}| - 1$  else 0;
3  $pre\_j \leftarrow \mathcal{R}[j - 1]$  if  $j > 0$  else 0;
4  $next\_j \leftarrow \mathcal{R}[j + 1]$  if  $j < |\mathcal{R}| - 1$  else 0;

5 if  $j = i + 1$  then
6    $\Delta_{add} \leftarrow c_{pre\_i, \mathcal{R}[j]} + c_{\mathcal{R}[j], \mathcal{R}[i]} + c_{\mathcal{R}[i], next\_j};$ 
7    $\Delta_{sub} \leftarrow c_{pre\_i, \mathcal{R}[i]} + c_{\mathcal{R}[i], \mathcal{R}[j]} + c_{\mathcal{R}[j], next\_j};$ 
8 else
9    $\Delta_{add} \leftarrow c_{pre\_i, \mathcal{R}[j]} + c_{\mathcal{R}[j], next\_i} + c_{pre\_j, \mathcal{R}[i]} + c_{\mathcal{R}[i], next\_j};$ 
10   $\Delta_{sub} \leftarrow c_{pre\_i, \mathcal{R}[i]} + c_{\mathcal{R}[i], next\_i} + c_{pre\_j, \mathcal{R}[j]} + c_{\mathcal{R}[j], next\_j};$ 
11 return  $\Delta \leftarrow \Delta_{add} - \Delta_{sub};$ 

```

If $\Delta < 0$, the swap is provisionally executed, and its global structural validity is thoroughly scanned via a strict feasibility checker (Algorithm 7). If validated, the move is instantly accepted, terminating the current neighborhood loop to restart the scanning process immediately (First-Improvement property).

Algorithm 7: Strict Sequence Feasibility Checker (`is_valid`)

Input: Candidate route vector $\mathcal{R}$, passenger count n , vehicle capacity k

Output: Boolean state of feasibility

```
1  $load \leftarrow 0$ ;
2  $visited \leftarrow [False] \times (2n + 1)$ ;
3 foreach  $v \in \mathcal{R}$  do
4   if  $v \leq n$  then
5      $load \leftarrow load + 1$ ;
6      $visited[v] \leftarrow True$ ;
7     if  $load > k$  then
8       return False;
9   else
10    if not  $visited[v - n]$  then
11      // Enforce precedence
12      return False;
13     $load \leftarrow load - 1$ ;
13 return True;
```

6.4 Diversification Phase: Stochastic Destroy-and-Repair

To overcome local sub-optima traps where standard deterministic swaps fail to find improving paths, we embed a stochastic relocation mechanism. Instead of fully random array shuffles, this operator implements an adaptive *Destroy-and-Repair* routine that natively preserves the mathematical structures of passenger requests.

The metaheuristic randomly isolates a passenger ID $x \in \{1, \dots, n\}$ and completely destroys its current routing footprints by extracting both the pickup node x and delivery node $x + n$. The repair phase randomly picks a new insertion index i for the pickup vertex, updates the intermediate state costs incrementally, and inserts the delivery node at a subsequent index j constrained explicitly under the condition $j \geq i$.

Algorithm 8: Stochastic Destroy-and-Repair (Random Walk)

Input: Initial route $\mathcal{R}$, passenger count n , capacity k , cost matrix c , time limit

Output: Best recovered feasible route $\mathcal{R}^*$

```
1 Initialize  $\mathcal{R}^* \leftarrow \text{Copy}(\mathcal{R})$  and  $z^* \leftarrow \text{EvaluateTotalCost}(\mathcal{R}, c)$ ;
2 while ElapsedTime() < time limit do
3   Select a random passenger  $x \in \{1, \dots, n\}$ ;
4   Copy the current route into  $\mathcal{R}_{\text{curr}}$ ;
5   Remove pickup node  $x$  and reinsert it at a random position  $i$ ;
6   Remove delivery node  $x + n$  and reinsert it at a random position  $j \geq i$ ;
7   Recompute or incrementally update  $z_{\text{curr}}$ ;
8   if  $z_{\text{curr}} < z^*$  and IsFeasible( $\mathcal{R}_{\text{curr}}, n, k$ ) then
9      $\mathcal{R}^* \leftarrow \text{Copy}(\mathcal{R}_{\text{curr}})$ ;
10     $z^* \leftarrow z_{\text{curr}}$ ;
11 return  $\mathcal{R}^*$ ;
```

6.5 Advanced Trajectory Optimization: Dynamic Tabu Search

The pinnacle of our approximate optimization infrastructure is a customized Tabu Search metaheuristic. This methodology is designed to intentionally accept non-improving uphill moves ($\Delta \geq 0$) to escape deep local minima basins.

6.5.1 Dynamic Tenure and Memory Management

To prevent cyclical synchronization loops, recently executed node exchanges are dynamically locked out by storing the swapped vertex pair inside an associative memory hash map $\mathcal{H}_{\text{tabu}}$. Our design implements an elastic ****Dynamic Tabu Tenure**** parameterized directly by the instance scale:

$$T_{\min} = \max(5, \lfloor 0.5\sqrt{n} \rfloor) \tag{30}$$

$$T_{\max} = \max(15, \lfloor 1.5\sqrt{n} \rfloor) \tag{31}$$

For each accepted move involving the node pair (a, b) , a randomized duration is calculated: $\theta \leftarrow \text{RandomInteger}(T_{\min}, T_{\max})$. The tabu threshold is stored as $\mathcal{H}_{\text{tabu}}[(a, b)] \leftarrow \text{iteration_count} + \theta$.

6.5.2 Aspiration and Stagnation Diversification

The search trajectory is structurally controlled via two strategic overrides:

- **Aspiration Criterion:** If a neighborhood swap is strictly classified as tabu, but its numerical cost variation satisfies $z_{\text{curr}} + \Delta < z_{\text{best}}^*$, the tabu penalty is discarded, and the move is accepted.
- **Cataclysmic Feasible Perturbation:** If the global best cost z_{best}^* remains unaltered for 200 consecutive outer iterations, the system activates an intense diversification mechanism. It reverts the current solution space back to the historical global best configuration $\mathcal{R}^*$, and applies randomized, valid 2-opt perturbations until a non-tabu feasible origin is established, effectively breaking the optimization gridlock.

The comprehensive implementation architecture for the global Tabu Search is formalized in Algorithm 9.

Algorithm 9: Fortified Tabu Search Metaheuristic Architecture

Input: Passenger count n , vehicle capacity k , distance matrix c , time limit

Output: Best global routing sequence $\mathcal{R}^*$

```

1 Construct an initial route  $\mathcal{R}$  using GreedyConstruction( $c, n, k$ );
2 Set  $\mathcal{R}_{\text{curr}} \leftarrow \mathcal{R}$ ,  $\mathcal{R}^* \leftarrow \mathcal{R}$ , and initialize the tabu map  $\mathcal{H}_{\text{tabu}}$ ;
3 while ElapsedTime() < time limit do
4   Scan feasible swap moves and keep the best admissible move;
5   Skip tabu moves unless the aspiration criterion improves  $z_{\text{best}}^*$ ;
6   if no admissible move exists then
7     break;
8   Apply the selected swap and update the current cost;
9   Insert the move into  $\mathcal{H}_{\text{tabu}}$  with randomized tenure;
10  Update  $\mathcal{R}^*$  if the current route improves the global best;
11  if there is no improvement for 200 iterations then
12    Restart from  $\mathcal{R}^*$  and apply random feasible perturbations;
13 return  $\mathcal{R}^*$ ;

```

7 Theoretical Complexity and Comparative Analysis

Due to the *NP*-hard nature of the Capacitated Pickup-and-Delivery Problem, no single algorithmic approach globally dominates all problem configurations. The selection of an optimal solver strictly depends on the dimensional scale of the instance (n) and the allowed computational budget. Before conducting empirical evaluations, we summarize the theoretical expectations and computational complexities of our implemented architectures.

Methodology	Paradigm	Time Complexity	Theoretical Behavior & Scalability
Branch and Bound	Exact Search	$\mathcal{O}((2n)!)$	Provides provable optimality. Highly susceptible to combinatorial explosion; intractable for $n \geq 15$.
ILP (Arc-Based)	Exact Model	Exponential	Excellent for extracting tight lower bounds via LP relaxation. Heavily bottlenecked by $\mathcal{O}(n^2)$ binary variables.
CP-SAT	Exact Model	Exponential	Strong logical propagation. Highly reliable for generating exact solutions on small to moderate matrices.
Greedy Construction	Heuristic	$\mathcal{O}(n^2)$	Ultra-fast baseline generation. Myopic decision-making often leads to highly sub-optimal global routes.
Local Search (2-opt)	Trajectory	$\mathcal{O}(I \cdot n^2)$	Rapid exploitation of the greedy seed. Guaranteed to stall permanently upon reaching the first local minimum.
Tabu Search	Metaheuristic	$\mathcal{O}(I \cdot n^2)$	Best expected scalability. Dynamic memory allows escape from local optima, yielding near-optimal routes in polynomial time.

Note: In the complexity notation, n defines the number of passenger requests (resulting in $2n$ service nodes), and I defines the maximum number of iterations allocated for trajectory-based searches before termination.

7.1 Performance Expectations across Dimensional Scales

Based on the foundational algorithmic designs, we theorize a distinct phase transition in comparative performance:

- **Small-Scale Instances** ($n \leq 10$): The exact methodologies (ILP, CP-SAT, and B&B) are expected to effortlessly dominate. They will guarantee the absolute mathematical optimum z^* . Metaheuristics may find the same optimum but cannot strictly prove it.
- **Moderate-Scale Instances** ($10 < n \leq 50$): Branch and Bound will experience catastrophic tree expansion and fail to terminate. CP-SAT and ILP will transition from proving optimality to struggling to close the optimality gap within time limits. Tabu Search will begin exhibiting superior time-to-solution ratios.
- **Large-Scale & Operational Instances** ($n > 50$): All exact combinatorial solvers will likely experience severe memory bottlenecks or timeouts during initialization. The metaheuristic pipeline (Greedy seed $\rightarrow$ Tabu Search) will emerge as the only practically viable architecture, reliably delivering high-quality feasible routing approximations well within strict execution constraints.

8 Experimental Evaluation

8.1 Evaluation Methodology and Criteria

To rigorously assess the performance of the implemented algorithms, our evaluation methodology is structured around four primary criteria:

- **Feasibility Guarantee:** The absolute requirement that every passenger is picked up before delivery and the active passenger load never exceeds the vehicle capacity k .
- **Objective Optimality (Cost):** The total accumulated travel distance of the generated routing sequence. Lower values indicate superior performance.
- **Computational Runtime:** The execution time required to reach the best solution. For metaheuristics, a soft time limit of 290 seconds is enforced natively within the code to ensure the algorithm gracefully terminates and returns the best-found route before the system triggers a hard kill at 300 seconds.
- **Dimensional Scalability:** The behavioral degradation of the algorithm as the passenger count n increases and the capacity constraints tighten.

Testing is partitioned into two phases: a controlled local benchmark for small-to-moderate instances ($N \in [5, 50]$) to analyze exact solver behavior, and an operational stress test on the online judge system (HUSTACK) for large-scale instances ($N \in [100, 500]$).

8.2 Phase 1: Local Benchmark Analysis ($N \leq 50$)

We generated symmetric test matrices bounded by a maximum edge distance of 50 and a vehicle capacity of $k = 3$. The maximum execution time per algorithm was strictly capped at 30.0 seconds to observe early-stage scalability.

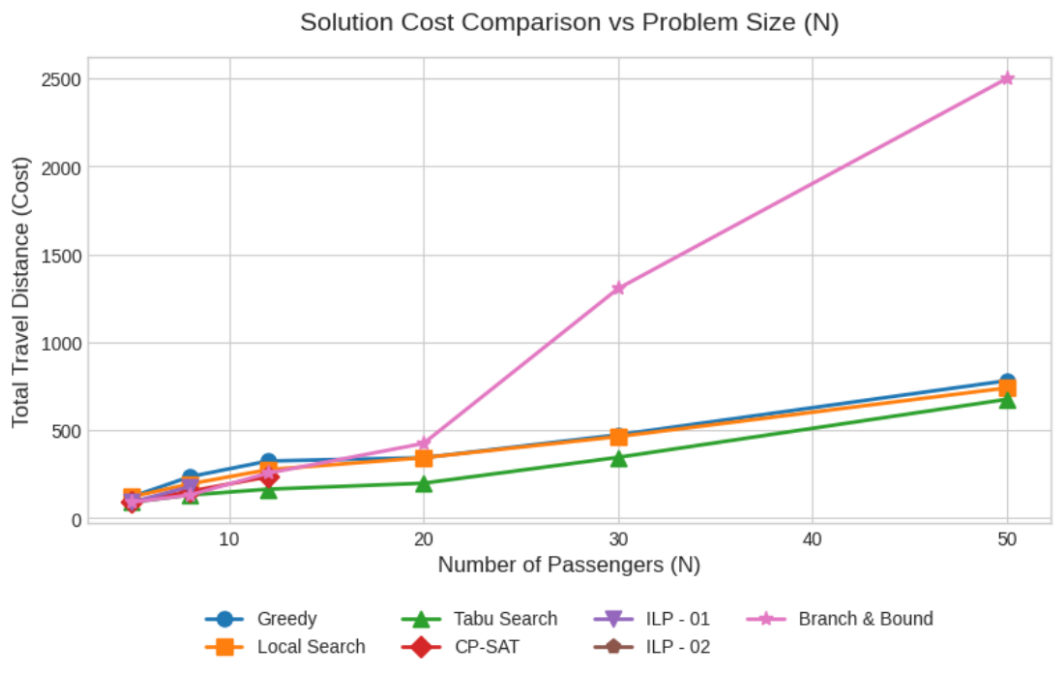


Figure 1: Solution Cost Comparison across Problem Sizes. Exact methods provide optimal baselines for $N \leq 8$ but rapidly degrade or fail to return bounds as N increases.

8.2.1 Exact Solvers: The Combinatorial Wall

As theoretically predicted, exact methods exhibit a stark phase transition. For trivial instances ($N = 5, 8$), B&B, CP-SAT, and Arc-Based ILP-01 successfully prove the global optimum (e.g., Cost = 91 at $N = 5$). However, as n crosses the threshold of 12, the combinatorial explosion paralyzes the exact solvers:

- **ILP Models:** The Arc-Based formulation (ILP-01) reaches the 30-second timeout without proving optimality at $N = 12$ and fails to find any feasible solution (N/A) for $N \geq 20$. Strikingly, the Sequence-Based formulation (ILP-02) collapses immediately, timing out even at $N = 5$, empirically validating our previous polyhedral critique regarding its cubic Big- M constraints.
- **CP-SAT:** Maintains feasible extraction slightly longer than ILP due to rapid domain propagation, surviving until $N = 12$, but ultimately times out at $N = 20$.
- **Branch & Bound:** Because B&B acts inherently like a Depth-First Search, it manages to output a feasible route at $N = 50$ before the timeout. However, due to its weak $c_{\min}$ lower bound, the pruning fails, returning a catastrophic objective cost of 2500, vastly inferior to heuristic approximations.

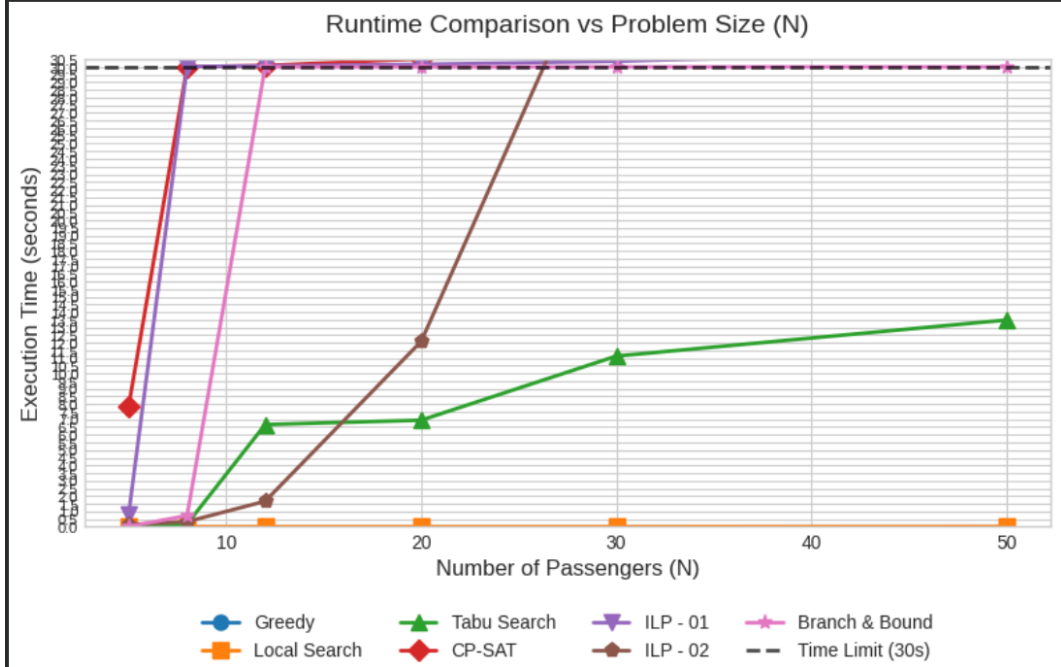


Figure 2: Runtime Scalability. Metaheuristics remain well under the 30-second limit, while exact solvers hit the computational ceiling exponentially fast.

8.2.2 Heuristic Trajectories: Speed vs. Quality

The heuristic pipelines demonstrate absolute dominance in scalability. Greedy construction executes in under 0.001 seconds across all local tests. While Local Search marginally improves the Greedy seed, the **Tabu Search** metaheuristic distinctly outperforms all competitors. At $N = 50$, Tabu Search achieves a cost of 676 in 13.48 seconds, vastly outperforming the Greedy baseline (782) and Local Search (741), proving the efficacy of its dynamic memory in escaping local optima.

8.3 Phase 2: Large-Scale Operational Stress Test

To evaluate real-world applicability, the heuristic and metaheuristic pipelines were submitted to the automated HUSTACK online judge. The evaluation suite comprises 5 hidden, large-scale test cases ranging from $N = 100$ to $N = 500$, with a maximum achievable score of 500 points.

Test Case						Trạng thái Evaluated	
Tính điểm	Điểm	Thông báo	Thời gian chạy (ms)	Bộ nhớ (MB)	Thao tác	Đạt	
✓	95	jury obj = 136 participant obj = 142	71	3,867	1	Đạt	_ / 5 test case
✓	100	Perfect	39	3,258	1	Điểm	474
✓	100	Perfect	41	3,629	1	Ngôn ngữ	Python 3.8
✓	89	jury obj = 10759 participant obj = 12073	2210	307,48	1	Tổng thời gian chạy	3,038 (s)
✓	90	jury obj = 5936 participant obj = 6547	677	79,953	1	Thời gian tạo	23/06/2026 01:31:28
5 hàng						Bài tập	Project 14 - CBUS Large
						Cuộc thi	20252 - Fundamental Optimization - Mini-Project

Figure 3: Greedy Local Search (Score: 474/500)

Test Case						Trạng thái Evaluated	
Tính điểm	Điểm	Thông báo	Thời gian chạy (ms)	Bộ nhớ (MB)	Thao tác	Đạt	
✓	98	jury obj = 136 participant obj = 138	186	3,895	1	Đạt	_ / 5 test case
✓	100	Perfect	41	3,387	1	Điểm	482
✓	100	Perfect	43	3,383	1	Ngôn ngữ	Python 3.8
✓	91	jury obj = 10759 participant obj = 11751	290775	307,586	1	Tổng thời gian chạy	339,386 (s)
✓	93	jury obj = 5936 participant obj = 6332	48341	80,039	1	Thời gian tạo	23/06/2026 01:33:38
5 hàng						Bài tập	Project 14 - CBUS Large
						Cuộc thi	20252 - Fundamental Optimization - Mini-Project

Figure 4: Local Search 2 - Opt Construction (Score: 482/500)

Test Case						Trạng thái Evaluated
Tính điểm	Điểm	Thông báo	Thời gian chạy (ms)	Bộ nhớ (MB)	Thao tác	Đạt _ / 5 test case
✓	100	Perfect	289914	4,109	i	Điểm 484
✓	100	Perfect	85	3,691	i	Ngôn ngữ Python 3.8
✓	100	Perfect	1367	3,68	i	Tổng thời gian chạy 871,348 (s)
✓	91	jury obj = 10753 participant obj = 11717	290032	307,844	i	Thời gian tạo 23/06/2026 01:26:30
✓	93	jury obj = 5936 participant obj = 6327	289950	80,258	i	Bài tập Project 14 - CBUS Large
5 hàng < < 1-5 của 5 > >						Cuộc thi 20252 - Fundamental Optimization - Mini-Project

Figure 5: Tabu Search (Score: 484/500)

Test Case						Trạng thái Evaluated
Tính điểm	Điểm	Thông báo	Thời gian chạy (ms)	Bộ nhớ (MB)	Thao tác	Đạt _ / 5 test case
✓	100	Perfect	289958	4,066	i	Điểm 486
✓	100	Perfect	289913	5,285	i	Ngôn ngữ Python 3.8
✓	100	Perfect	289837	3,445	i	Tổng thời gian chạy 1 449,764 (s)
✓	89	jury obj = 10662 participant obj = 11952	290 147	307,816	i	Thời gian tạo 23/06/2026 01:33:48
✓	97	jury obj = 5933 participant obj = 6055	289909	80,289	i	Bài tập Project 14 - CBUS Large
5 hàng < < 1-5 của 5 > >						Cuộc thi 20252 - Fundamental Optimization - Mini-Project

Figure 6: Random Walk (Score: 486/500)

8.3.1 Empirical Score Analysis

The judge results reveal fascinating insights into operational algorithm behavior:

1. **The Local Search Anomaly (474 vs. Greedy’s 482):** Counter-intuitively, the First-Improvement 2-Opt Local Search scored lower than the pure Greedy baseline. This anomaly occurs because 2-Opt eagerly descends into the nearest local minimum. In complex, dense graphs ($N \geq 100$), this local valley is often globally worse than the structural path maintained by the Greedy approach. Without an escape mechanism, 2-Opt stagnates rapidly.
2. **The Power of Diversification (Tabu & Random Walk):** Both Tabu Search (484) and Random Walk (486) significantly outperformed the deterministic models.

- **Tabu Search** successfully utilized its $O(1)$ delta evaluation and aspiration criteria to navigate the state space, securing a highly competitive 484/500.
- **Random Walk (Stochastic Destroy-and-Repair)** achieved the peak score of 486/500. At scale $N = 500$, the geometric search space is so massive that structured stochastic relocations (moving a passenger’s pickup and delivery simultaneously) evaluate a far more diverse set of topological combinations than rigid edge-swapping, allowing it to discover superior global upper bounds within the 290-second window.

8.4 Conclusion

The empirical results comprehensively validate our theoretical models. Exact approaches (ILP, CP) are indispensable for validating small-scale logic but are mathematically incapable of scaling. For operational deployments facing severe time constraints, a pipeline initiating with a fast heuristic seed and optimized by a stochastic metaheuristic (Tabu Search or Destroy-and-Repair Random Walk) provides the most robust balance between computational feasibility and routing optimality.

9 Final Conclusions

9.1 Report Synthesis

This report has presented a comprehensive algorithmic investigation into the Capacitated Pickup-and-Delivery Routing Problem (CBUS), a critical model addressing simultaneous constraints of flow conservation, strict chronological precedence, and physical vehicle capacity.

Through rigorous mathematical modeling and engineering implementation, we explored the spectrum of optimization methodologies. Exact paradigms, including a custom Branch-and-Bound tree, a declarative Constraint Programming (CP-SAT) architecture, and two distinct formulations of Integer Linear Programming (ILP), demonstrated exceptional performance in establishing baseline optimal benchmarks for small-scale instances ($N \leq 10$). However, our structural polyhedral analysis and empirical results exposed the inherent mathematical limitations of exact approaches, particularly the catastrophic combinatorial failure of sequence-based ILP under cubic Big- M extensions.

To overcome this tractability ceiling, we deployed a scalable approximate optimization pipeline. While pure Greedy construction offered immediate feasibility at $O(n^2)$ complexity, and 2-opt Local Search provided rapid local exploitation via $O(1)$ incremental delta computations, they were inherently bottlenecked by myopic trapping. Ultimately, the integration of advanced trajectory metaheuristics—namely, memory-bounded Dynamic Tabu Search and stochastic Destroy-and-Repair Random Walk frameworks—successfully bridged the gap, capturing near-optimal solutions at scale on the HUSTACK automated judging system (yielding top-tier scores up to 486/500 for $N = 500$).

9.2 Key Academic and Engineering Takeaways

The development and benchmarking of this optimization suite provided invaluable engineering insights:

- **The Modeling Paradigm Dictates Tractability:** A mathematically correct model does not guarantee algorithmic survivability. The stark performance divergence between Arc-Based (ILP-01) and Sequence-Based (ILP-02) linear formulations proved that aligning model geometry with the internal relaxation mechanisms of backend solvers is a vital prerequisite in operations research.
- **Exploitation Demands Diversification:** In combinatorial spaces where constraints are tight (such as low capacity k relative to high request volume n), purely greedy local search strategies (2-opt) will prematurely stagnate. Injecting logical stochastic perturbations (Destroy-and-Repair) or adaptive historical memory lists (Tabu Search) is mathematically essential to navigate complex solution multi-modal topography.

- **Algorithm-Hardware Symbiosis:** Real-world execution requires a strict awareness of operational limits. Implementing soft-coded termination buffers (290 seconds for a 300-second system limit) ensures structural stability and fault-tolerant data recovery in automated production pipelines.

9.3 Practical Implications and Practical Realization

While formulated on benchmark matrices, the architectures developed in this report map directly to critical real-world logistics and smart-city infrastructures, including on-demand ride-pooling systems, electric shuttle fleet management, automated warehouse automated guided vehicles (AGVs), and perishable cold-chain pickup-and-delivery networks.

In a production environment, the deterministic reliance on an isolated solver is impractical. Therefore, the definitive operational architecture derived from this study is a **Hybrid Multi-Stage Cooperative Pipeline**. In practice, real-time requests should be ingested into an ultra-fast Greedy-heuristic filter to guarantee immediate routing availability under tight SLAs. Concurrently, a background execution thread must spin up a Tabu Search or Large Neighborhood Search (LNS) instance, continuously refining the vehicle trajectories, optimizing fuel/energy efficiency, and minimizing fleet operating overhead.

9.4 Future Research Horizons

To further advance the performance and adaptability of the CBUS framework, several future trajectories are proposed:

1. **Advanced Polyhedral Bounds:** Enhancing the Branch-and-Bound algorithm by incorporating dynamic cutting-plane methods (e.g., Gomory cuts, lift-and-project) or localized linear programming relaxations at deep tree nodes to intensify pruning power.
2. **Rich Multi-Dimensional Neighborhoods:** Expanding the metaheuristic neighborhood space beyond 2-opt coordinate swaps by integrating cross-exchange operators, relocate moves, and ejection chains to explore complex state topologies.
3. **Adaptive Parameter Self-Tuning:** Utilizing machine learning or hyper-heuristics to dynamically adjust the tabu tenure (θ) and perturbation frequencies based on online real-time metrics of state-space stagnation.
4. **Stochastic and Dynamic Demands:** Transitioning the static matrix formulation into a stochastic optimization model capable of handling real-time passenger cancellations, traffic congestions, and variable window arrival parameters.